

Temporal Super-Sampling: Technical Report

Shiven Bajpai

1 Overview

This report documents the technical implementation of a temporal super-sampling network that upscales rendered game frames by a factor of $2\times$ ($540p \rightarrow 1080p$), using auxiliary channels provided by the game engine (depth, motion vectors, and sub-pixel jitter offsets) in addition to color. The network follows a single-frame recurrent design in the style of Mercier et al.[1], in which a per-pixel blend mask combines a freshly synthesized high-resolution candidate frame with a motion-warped version of the previous output. The model is trained on a subset of the Qualcomm Rasterized Images (QRISP) dataset [2].

2 Dataset

2.1 Source and modalities

A subset of the QRISP dataset (specifically the **ScifiBase** scenes) is used, containing paired low-resolution ($540p$, 960×540) and native high-resolution ($1080p$, 1920×1080) renders of the same sequences, along with auxiliary buffers rendered at the low resolution:

- **LR color:** jittered, mip-biased color buffer (**MipBiasMinus1Jittered**).
- **LR depth:** packed into an 8-bit RGBA PNG and unpacked in software into a single-channel float depth map via a base-255 weighted sum of the four channels.
- **LR motion vectors:** stored as an EXR buffer (**MotionVectorsMipBiasMinus1Jittered**).
- **Per-frame camera jitter:** sub-pixel (x, y) jitter offset read from a per-frame JSON camera-data file.
- **HR target:** native-resolution ground-truth color (**Native**).

2.2 Preprocessing

A preprocessor converts each scene’s per-frame PNG/EXR/JSON files into contiguous, memory-mapped float32 arrays (**lr_color.dat**, **depth.dat**, **motion.dat**, **jitter.dat**, **hr_color.dat**) on disk, one set of files per scene, to allow fast random access during training without repeated image decoding.

2.3 Splits and sampling

Scenes are split 70/15/15 into train/validation/test sets at the scene level. To reduce memory footprint of individual training samples, each scene is cropped as:

- a **temporal** crop of consecutive frames from the scene (14 frames for training, 7 for validation), and
- a **spatial** crop of the low-resolution frame (only applied on training set), with the matching $2\times$ crop applied to the high-resolution target.

3 Network Architecture

The architecture used is a direct implementation of the model described by Mercier et. al[1]. Refer to the paper for more details.

3.1 Model configuration used

Hyperparameter	Value
Recurrent feature channels	10
Hidden feature channels	48
Convolution blocks in backbone	3
Jitter-MLP hidden width	512
Motion-vector dilation block size	6

4 Training Setup

4.1 Loss

Training minimizes a composite objective over each predicted frame batch:

$$\mathcal{L} = \mathcal{L}_1 + 0.1 \cdot \mathcal{L}_{\text{LPIPS(AlexNet)}} + 0.2 \cdot (1 - \text{SSIM}),$$

using the standard L1 loss, the `lpips` package’s AlexNet-based perceptual metric, and the `pytorch_msssim` SSIM implementation. A Charbonnier loss class and a hand-written SSIM function are also defined in the codebase but are not the loss used by the training loop.

4.2 Optimization

- Optimizer: Adam, learning rate 2×10^{-4} .
- Scheduler: `StepLR`, step size 10 epochs, decay factor 0.5.
- Gradient norm clipping at 1.0.
- Batch size: 7 sequences; trained for 100 epochs.
- Sequences are processed in chunks of 7 frames to bound memory use; the recurrent state is detached and carried across chunks within a sequence, and trailing chunks shorter than 5 frames are dropped.

4.3 Metrics and validation

PSNR (computed as $10\log_{10}(1/\text{MSE})$ on the $[0, 1]$ -normalized color output) and the training loss are logged every epoch on the training set. A full validation pass runs every 10 epochs, logging validation loss/PSNR and exporting a qualitative debug image comparing bicubic-upsampled input, prediction, ground truth, and absolute error for the first validation batch.

4.4 Environment

The notebook was run on Google Colab (T4 GPU).

5 Results

5.1 Training curves



Figure 1: Training/validation loss and PSNR over epochs.

5.2 Qualitative comparison

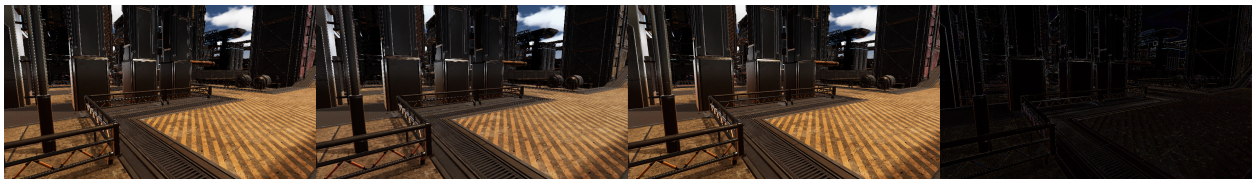


Figure 2: Side by side comparison, From left to right: bicubic-upsampled input, model prediction, ground truth, and absolute difference

5.3 PSNR summary

Split	PSNR (dB)
Train	23.67
Validation	22.41

Table 1: PSNR achieved by model on training and validation sets

6 Future work

- **Scaling up training.** The current model is only trained on a small subset of QRISP, due to limitations posed by the resource limits on Google Colab. Training the model on a larger dataset is likely to significantly improve performance
- **Fine tuning and deploying to custom environment.** Once the model is trained to satisfactory performance levels, it can be fine tuned to scenes from a particular game, and then can be compiled to GPU shaders or added to the graphics pipeline in some other way to create a live deployment of the system.
- **Comparing Architectures.** In this project we mainly took one existing work and tried to replicate it. Several models may be tested on the same dataset to see which performs better, and a better model may be used.

References

- [1] A. Mercier, R. Erasmus, Y. Savani, M. Dhingra, F. Porikli, and G. Berger. *Efficient Neural Supersampling on a Novel Gaming Dataset*. Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023, pp. 296–306. https://openaccess.thecvf.com/content/ICCV2023/papers/Mercier_Efficient_Neural_Supersampling_on_a_Novel_Gaming_Dataset_ICCV_2023_paper.pdf
- [2] Qualcomm Technologies, Inc. *Qualcomm Rasterized Images Dataset (QRISP)*. <https://www.qualcomm.com/developer/software/qualcomm-rasterized-images-dataset>